SMART QUEUE & CROWD PREDICTION SYSTEM

Java-Based Capstone Project Proposal

Prepared by: **A.B.M Isratul Bari(Wasi)**

Student ID: IT24056

Department: Information and Communication Technology (ICT)

Institution: Mawlana Bhashani Science and Technology University

Course: Software Enggineering

Submission Date: 27/10/26

Submitted to**: Dr. Ziaur Rahman**

Professor

Department of ICT,MBSTU

Executive Summary

The Smart Queue & Crowd Prediction System is a Java-based web application designed to reduce uncertainty and unnecessary waiting at service centres such as university offices, clinics, banks, and public service counters. Customers can request a digital token, monitor their queue position, view an estimated waiting time, and review expected crowd levels. Administrators can manage service counters, process tokens, and review historical service statistics. The first version will use a transparent mathematical waiting-time estimator and historical-average crowd forecasting. These methods are appropriate for a capstone prototype because they are explainable, testable, and do not require a large training dataset.

1. Introduction

1.1 Background

Many service centres rely on physical queues or basic token systems. Customers may not know how many people are ahead of them or how long service is likely to take. Managers may also lack an accessible summary of peak hours and counter performance. A web-based queue system can provide queue visibility and collect service history that supports operational planning.

1.2 Problem Statement

The project addresses four problems: uncertain waiting time, avoidable crowding, limited visibility of queue progress, and insufficient use of historical service data for planning. The system will provide digital queue tracking and approximate forecasts, while clearly communicating that predictions are estimates rather than guarantees.

1.3 Aim

To design and prototype a web-based queue management application that estimates customer waiting time and predicts crowd levels using Java and a relational database.

1.4 Objectives

- Provide role-based customer and administrator interfaces.
- Generate and manage digital queue tokens.
- Display current queue position and estimated waiting time.
- Estimate crowd levels from historical customer counts by time slot.
- Manage service counters, appointments, and service records.
- Produce summary reports and evaluate the accuracy of estimates.
- Document the solution with six required diagrams and a project schedule.

1.5 Scope

The initial prototype will support one service centre with multiple counters and configurable services. It will use manually entered or simulated historical data for crowd forecasts. Email/SMS integration, payment processing, multi-branch deployment, and production-grade machine learning are outside the first release unless time permits.

2. Stakeholders and Users

Stakeholder	Needs / Responsibilities
Customer	Request a token, check position and estimated waiting time, view crowd levels, book/cancel appointments.
Administrator	Manage services and counters, call the next token, mark service complete, review reports.
Service staff	Serve customers and update token status.
Project developer	Gather requirements, design, implement, test, document, and publish the project.
Instructor/evaluator	Review proposal quality, diagrams, prototype, tests, and repository.

3. Proposed Solution and Functional Requirements

ID	Functional Requirement	Priority
FR-01	The system shall allow users to register and log in.	Must
FR-02	A customer shall select a service and request a queue token.	Must
FR-03	The system shall assign a unique token number and issue time.	Must
FR-04	The system shall display token status and queue position.	Must
FR-05	The system shall estimate waiting time using queue and service data.	Must
FR-06	The system shall estimate crowd level for a future time slot.	Must
FR-07	An administrator shall manage service counters and token status.	Must
FR-08	The system shall store completed service records.	Must
FR-09	The system shall provide basic queue and crowd reports.	Should
FR-10	Customers shall be able to request an appointment.	Could

4. Non-Functional Requirements

- Usability: clear navigation and readable status indicators.
- Performance: common dashboard requests should normally complete within 3 seconds on a local development setup.
- Security: store password hashes, validate input, and restrict administrative actions by role.
- Reliability: persist token and service status in the database.
- Maintainability: separate controllers, service logic, domain models, and repositories.
- Portability: build with Maven and document setup steps.
- Privacy: collect only information needed for queue operation and restrict access to personal data.

5. Software Process Model

5.1 Selected Model: Agile Iterative Development

An Agile iterative process is selected because the project can be delivered in small increments and reviewed frequently. Each iteration produces a testable result. The plan uses weekly iterations, a prioritized task list, and milestone reviews. Requirements and forecasts may be refined after feedback and testing.

Iteration	Focus	Output
1	Requirements and planning	Approved scope, user stories, proposal draft
2	Analysis and design	Architecture, six diagrams, database schema
3	Core domain and persistence	Java classes and database operations
4	Customer workflow	Token generation, queue position, customer pages
5	Admin workflow	Counter controls, token processing, service records
6	Prediction and analytics	Waiting-time estimator, crowd forecast, charts/reports
7	Integration and verification	Integrated application, test report, fixes
8	Release and documentation	Final proposal, screenshots, README, GitHub repository

6. System Architecture and Technology

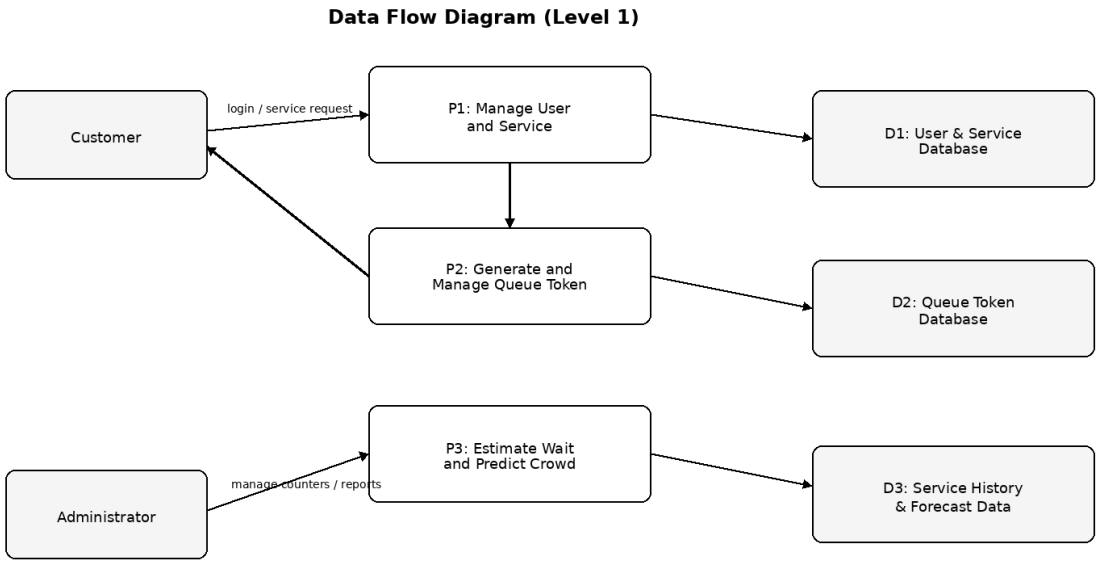
Architecture: three logical layers—presentation, service/business logic, and data access. The browser communicates with Spring Boot controllers; service classes apply queue and prediction rules; Spring Data JPA persists entities to MySQL.

Layer	Technology / Responsibility
Presentation	Thymeleaf, HTML, CSS, Bootstrap; customer and admin screens.
Application	Java 17+, Spring Boot; controllers and business services.
Persistence	Spring Data JPA / Hibernate; entity mapping and repository operations.
Database	MySQL; users, services, counters, tokens, appointments, records, forecasts.
Build / version control	Maven, Git, GitHub.
Testing	JUnit; unit tests for waiting-time and crowd classification logic.

7. Required System Diagrams

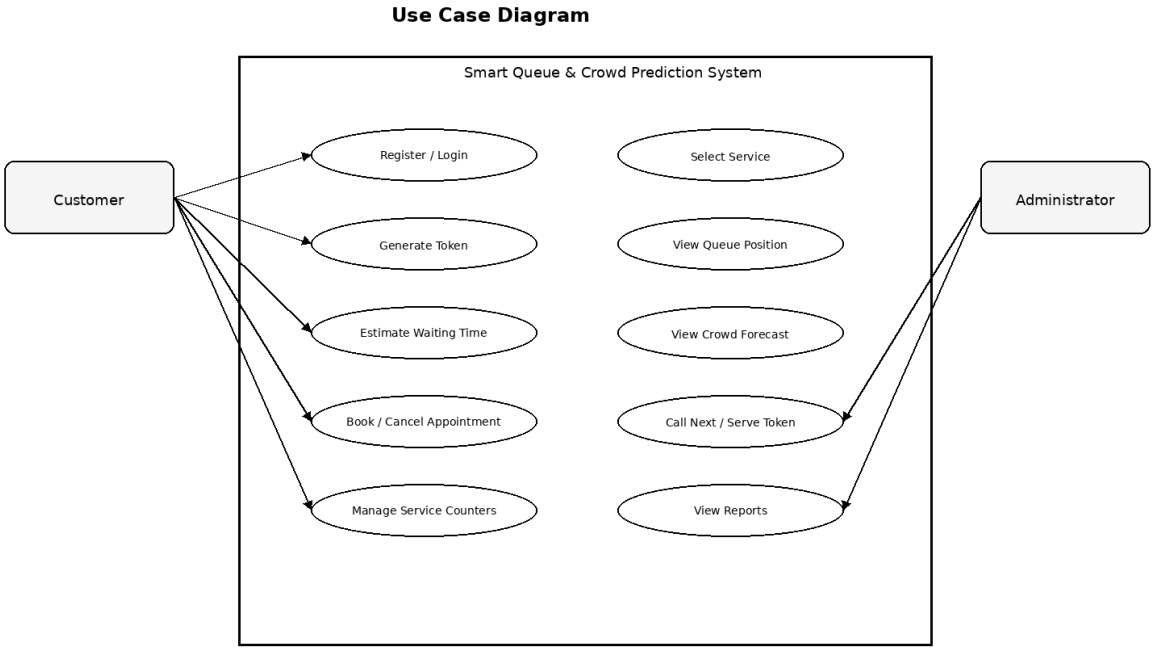
The diagrams below are initial proposal-level models. They should be reviewed during implementation so that the names and relationships remain consistent with the final code and database.

7.1 Data Flow Diagram (Level 1)



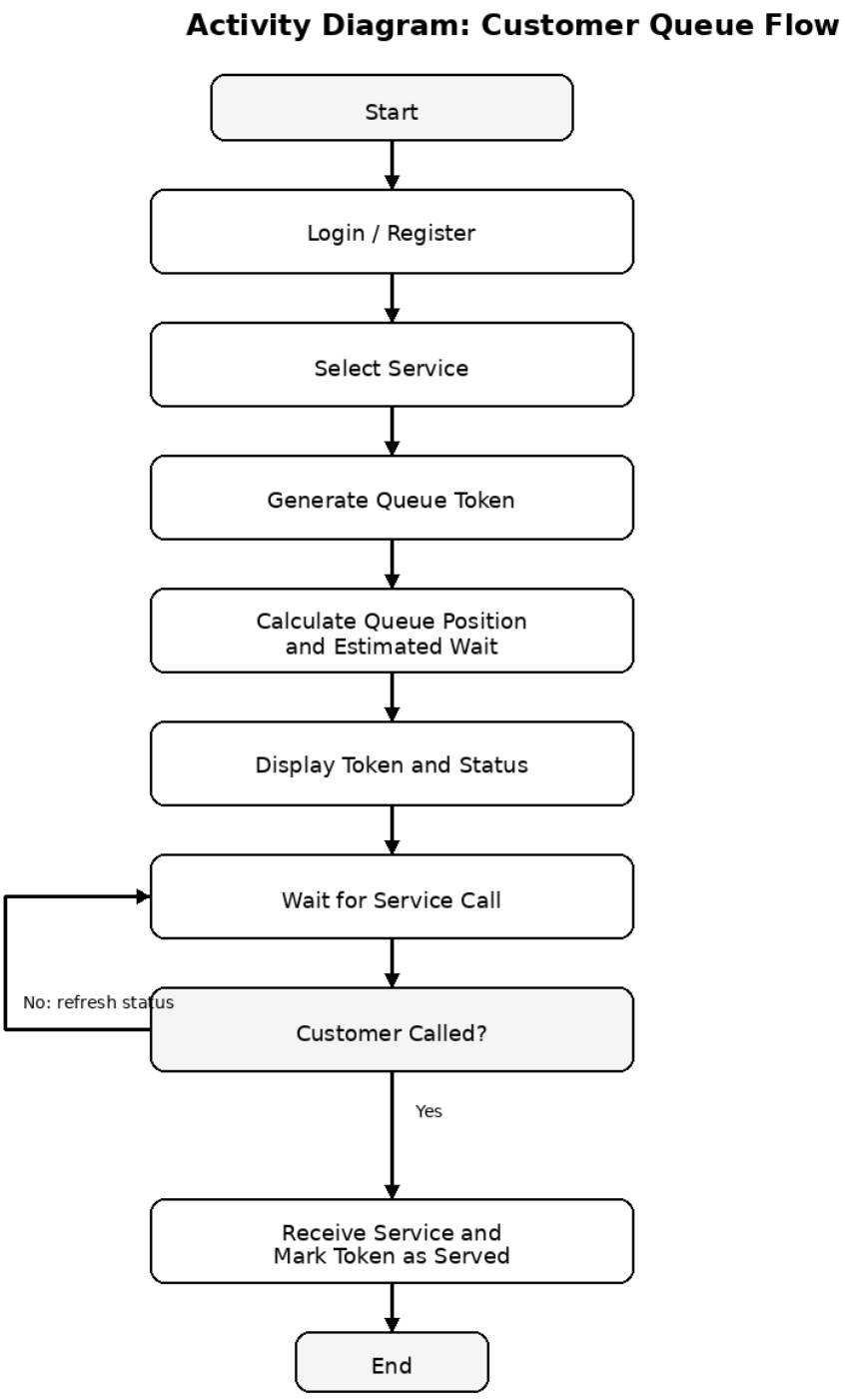
Customer and administrator inputs flow through account/service management, queue management, and prediction processes to the system data stores.

7.2 Use Case Diagram



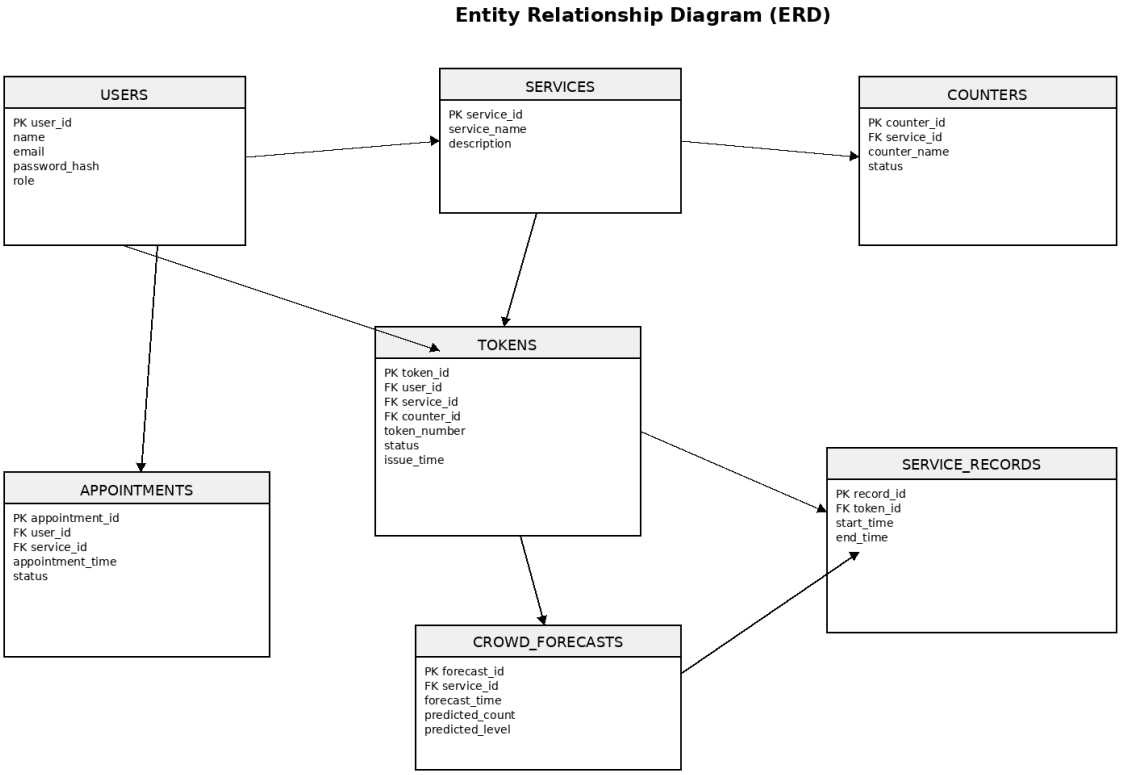
Shows customer and administrator interactions with the system.

7.3 Activity Diagram



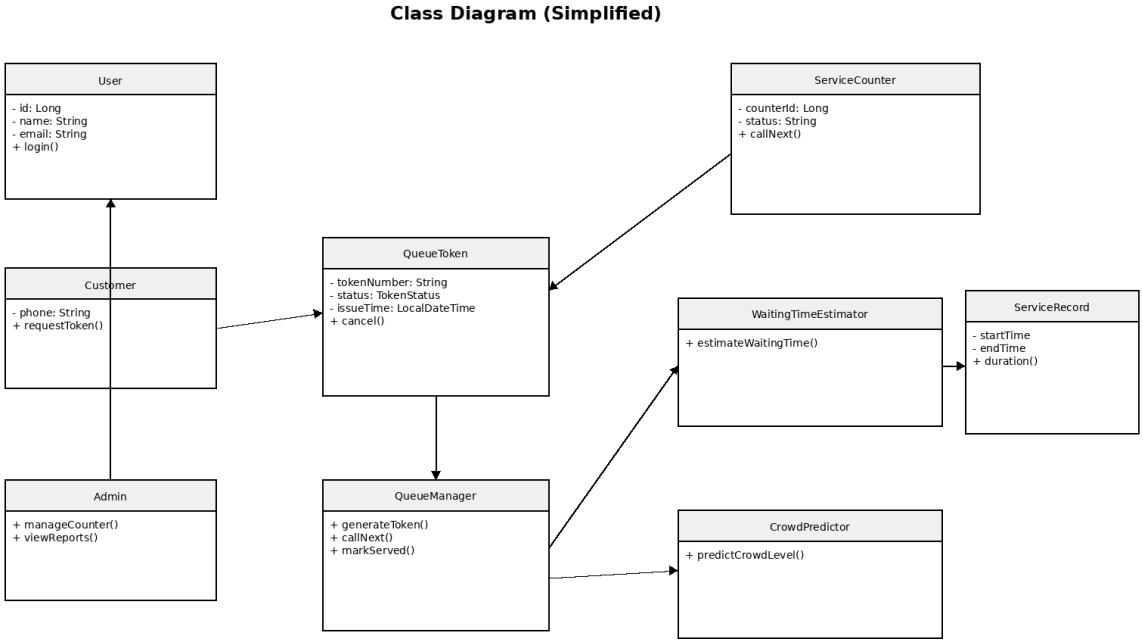
Shows the customer token journey from login through service completion.

7.4 Entity Relationship Diagram



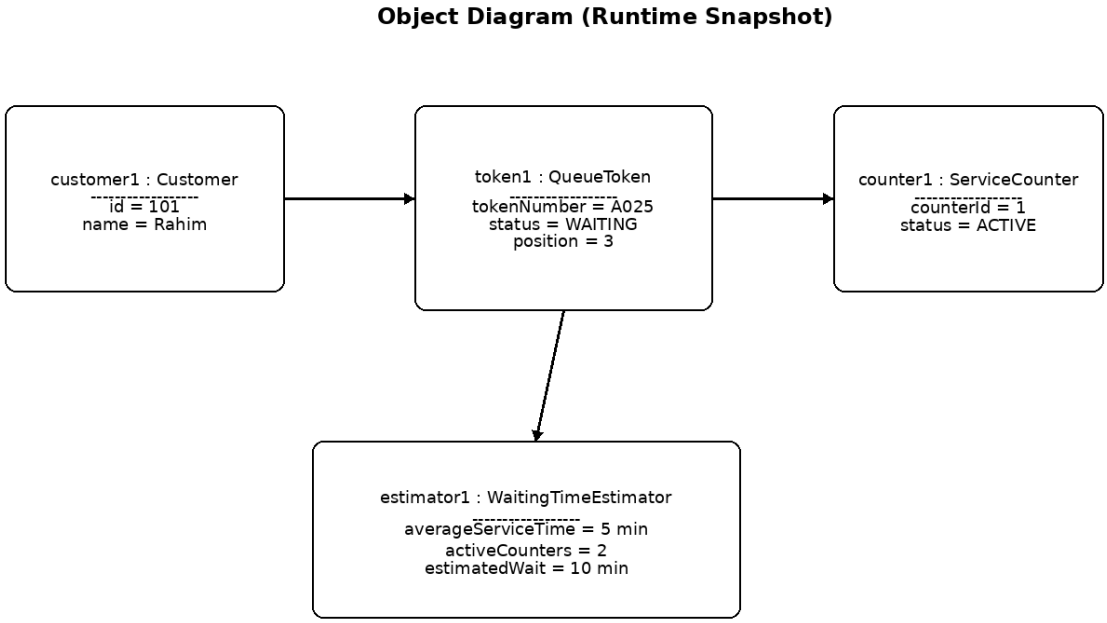
Shows the proposed relational entities and key relationships.

7.5 Class Diagram



Shows the principal Java classes, example attributes/methods, inheritance, and associations.

7.6 Object Diagram



Shows a runtime snapshot with a customer, token, counter, and waiting-time estimator.

8. Proposed Java Classes

Class	Responsibility
User	Base user identity and common account properties.
Customer extends User	Customer actions such as requesting a token.
Admin extends User	Administrative actions and reporting access.
ServiceCounter	Represents a service counter and its availability.
QueueToken	Token number, status, issue time, customer/service/counter links.
Appointment	Optional scheduled service request.
QueueManager	Token generation, ordering, calling, cancelling, and serving.
WaitingTimeEstimator	Calculates an approximate waiting time.
CrowdPredictor	Forecasts expected crowd using historical time-slot counts.
ServiceRecord	Stores start/end times and completed service details.
NotificationService	Provides in-app status messages.
ReportGenerator	Builds summaries from service records and forecasts.

9. Algorithm Design

9.1 Waiting-Time Estimation

Baseline formula: W = ceil(N × S / C), where N is the number of people ahead, S is average service duration in minutes, and C is the number of active counters serving the same queue. Example: N=12, S=5 minutes, C=3 gives W=20 minutes. The formula assumes reasonably balanced workload and similar counter speeds; the interface must label the result as an estimate.

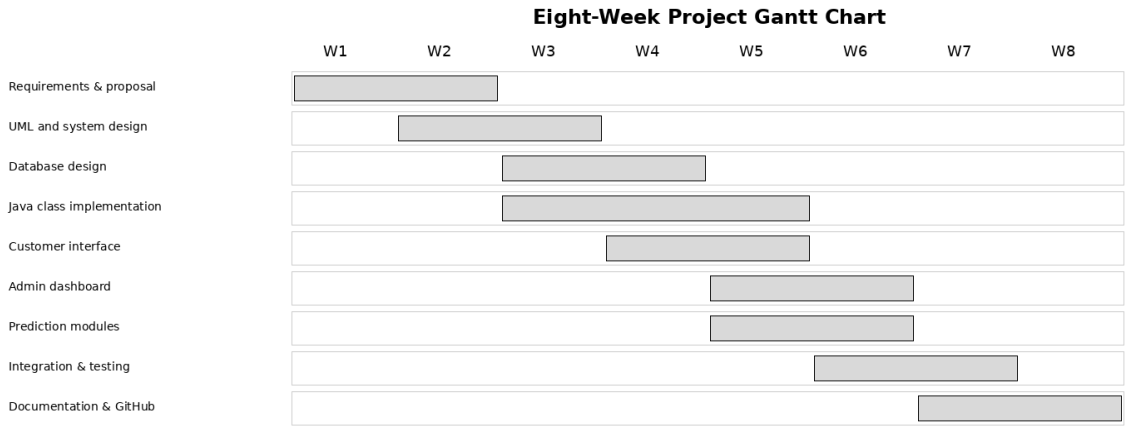
9.2 Crowd Forecasting

For each service and time slot, calculate the average number of customers recorded in comparable historical slots. Classify the forecast using configurable thresholds—for example, under 20 customers = Low, 20–39 = Medium, and 40 or more = High. These example thresholds must be tuned to the actual service centre. Evaluate forecasts by comparing predicted and observed counts.

10. User Interface Prototype

The initial interface will contain at least five class-backed pages/components: Customer Dashboard (Customer, QueueToken, QueueManager, WaitingTimeEstimator, CrowdPredictor); Token Booking (Customer, Appointment, QueueManager); Queue Status (QueueToken, ServiceCounter, WaitingTimeEstimator); Admin Dashboard (Admin, ServiceCounter, QueueManager, ServiceRecord, ReportGenerator); and Crowd Analytics (CrowdPredictor, ServiceRecord, ReportGenerator).

11. Project Schedule, Milestones, Deliverables



Milestone	Target	Deliverable / Acceptance Evidence
M1: Planning complete	End of Week 1	Proposal draft, scope, requirements list
M2: Design complete	End of Week 3	DFD, use case, activity, ERD, class, object diagrams

M3: Core model complete	End of Week 4	Java domain classes and database schema
M4: User workflows complete	End of Week 5	Customer token and queue pages
M5: Admin and prediction complete	End of Week 6	Admin controls, estimator, crowd forecast
M6: Verification complete	End of Week 7	Test cases, test results, issue fixes
M7: Submission ready	End of Week 8	Proposal, source code, screenshots, README, GitHub URL

12. Testing Strategy

Test ID	Scenario	Expected Result
T-01	Generate token for valid service	Unique token is saved and displayed.
T-02	Estimate wait for 12 people, 5 minutes, 3 counters	Baseline estimate is 20 minutes.
T-03	Active counter count is zero	Validation prevents division by zero and returns a clear error.
T-04	Admin calls next token	Earliest eligible waiting token becomes CALLED.
T-05	Admin marks called token served	Status and service record are persisted.
T-06	Historical count is below low threshold	Forecast label is LOW.
T-07	Customer attempts admin-only action	Access is denied.
T-08	Invalid or missing service selected	User receives validation feedback.

13. Risks, Limitations, and Mitigation

Risk / Limitation	Mitigation
Limited historical data	Use clearly labelled sample data for prototype; report forecast limitations.
Waiting-time estimates vary by service	Use per-service averages and update them from service records.
Scope becomes too large	Prioritize token workflow, queue status, admin controls, and baseline forecasts.
Security weaknesses	Use password hashing, role-based access, input validation, and environment-based secrets.
External notification services unavailable	Use in-app notifications in the initial release.

14. Expected Outcomes

- A working Java web prototype for digital queue management.
- A queue status view with approximate waiting time.
- A historical-average crowd forecasting component.
- An administrator dashboard and basic reports.
- A documented database and six consistent diagrams.
- A GitHub repository with source code, setup instructions, screenshots, and proposal.

15. Future Enhancements

- SMS or email notifications.
- Multi-branch and multi-service-centre support.
- More advanced forecasting models when sufficient data is available.
- QR-code token verification.
- Live charts and service-level performance indicators.
- Accessibility and multilingual interface support.

16. Conclusion

The Smart Queue & Crowd Prediction System combines practical queue operations with explainable analytics. Its scope is appropriate for an individual capstone when implemented incrementally. The proposed Agile plan, class design, database model, six diagrams, interface pages, and testing plan provide a structured path from proposal to demonstrable prototype.

Appendix A: Repository Checklist

- README with project overview and setup instructions.
- Maven source project and database schema/seed data.
- Six diagrams and Gantt chart.
- Screenshots of customer and administrator pages.
- Unit tests and test results.
- Proposal PDF or DOCX.
- GitHub repository URL added to the submission form.

Appendix B: References

- Oracle Java Documentation: https://docs.oracle.com/en/java/
- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/
- Spring Data JPA Reference: https://docs.spring.io/spring-data/jpa/reference/
- MySQL Documentation: https://dev.mysql.com/doc/
- OMG UML Specification: https://www.omg.org/spec/UML/